Rayls II: Fast, Private, and Compliant CBDCs

Mario Yaksetig[1 [0000−0003−2175−1957]], Pedro Pereira[1 [0000−0002−6951−6588]], Stephen Yang[1 [0009−0001−4557−2075]], Mahdi Nejadgholi[1 [0009−0000−4081−324X]], and Jiayu Xu[2 [0000−0002−0881−9980]]

[1] Parfin
mario.yaksetig@parfin.io
[2] Oregon State University, USA
xujiay@oregonstate.edu

Abstract. In this work, we introduce an upgrade to Rayls [1], a novel central bank digital currency (CBDC) design that provides privacy, high performance, and regulatory compliance. In the Rayls construction, commercial banks each run their own (private) ledger and are connected to an underlying decentralized programmable blockchain via a relayer. We introduce an improved and more efficient protocol that allows for efficient anonymous transactions between banks, which is (still) called Enygma. Our design is 'quantum-private' as a quantum adversary is not able to infer any information (i.e., payer, payee, amounts) about the transactions that take place in the network. One of the main improvements of this work is that the design is now completely quantum-secure except for the current use of ZK-SNARKs. We note, however, that this is modular and can simply be updated as desired.
We achieve high performance in cross-bank settlement via the use of ZK-SNARKs and 'double' batching and an optimized ZK implementation, with a prover time three times faster than the initial implementation and a cheaper on-chain verifier. Our transactions consist of a set of commitments and a zero-knowledge proof. As a result, each transaction can pay more than 1 bank at once and, secondly, each of these individual commitments can contain aggregated transfers from multiple users. For example, bank A transfers $1M to a different bank B and that amount is actually a sum of multiple users making transfers to bank B. Commercial banks can then enforce regulatory rules locally within their ledgers. Our system is in production with one of the largest clearing houses in the world and is currently being explored in a CBDC pilot.

Keywords: CBDCs · Privacy · Blockchain

1 Introduction

Central Bank Digital Currencies (CBDCs) represent a transformative advancement in monetary systems, offering digital representations of sovereign fiat currencies that maintain centralized control under national monetary authorities. As nations worldwide accelerate their exploration of CBDC implementations, the

technical challenges of achieving simultaneous privacy, scalability, auditability, and regulatory compliance have become increasingly apparent. The fundamental tension between providing transaction privacy comparable to physical cash while maintaining the transparency necessary for regulatory oversight presents a complex design space that existing solutions have struggled to navigate effectively.

The emergence of quantum computing capabilities introduces an additional layer of complexity to CBDC design considerations. As quantum algorithms threaten the cryptographic foundations of current privacy-preserving technologies, monetary authorities must consider quantum-resistant approaches when designing digital currency systems intended to operate for decades. This quantum threat is particularly relevant for privacy mechanisms, where the confidentiality of historical transactions must be preserved even against future quantum adversaries.

Recent work by the original Rayls design introduced a novel approach to CBDC architecture that addresses many of these challenges through a hybrid system where commercial banks operate private ledgers connected to an underlying decentralized blockchain via relayer mechanisms. This construction demonstrated promising results in unifying liquidity across institutions while maintaining privacy and regulatory compliance. However, the original Rayls protocol exhibited certain implementation efficiency limitations, particularly in the anonymous transaction mechanisms between financial institutions, and relied on elliptic curve cryptography (ECC) based bank identification in the Enygma protocol that may not provide adequate long-term security against quantum adversaries.

The scalability requirements for national-scale CBDC deployment remain formidable. Consider that major commercial banks in populous nations serve hundreds of millions of clients, with transaction volumes that exceed the current capabilities of any existing blockchain infrastructure. Traditional layer-2 scaling solutions, while offering some relief, introduce problematic liquidity fragmentation and privacy compromises when sequencers publish transaction data in cleartext to underlying layer-1 blockchains. These limitations underscore the need for fundamentally different architectural approaches that can achieve both horizontal scalability and privacy preservation.

Privacy preservation in CBDC systems faces unique constraints compared to traditional cryptocurrency applications. Citizens reasonably expect transaction privacy comparable to physical cash, yet regulatory authorities require mechanisms for compliance monitoring and auditing. Existing privacy solutions often impose linear computational costs for zero-knowledge proof generation or result in prohibitively large transaction sizes, making them impractical for national-scale deployment. The challenge intensifies when considering that privacy guarantees must hold against both classical and quantum adversaries over extended time horizons.

In this work, we present improvements to Rayls that address the implementation efficiency and some quantum security limitations of the original design.

Our enhanced protocol introduces optimized implementation performance for the anonymous transaction mechanism between financial institutions while transitioning from ECC-based to hash-based bank identification in the Enygma protocol to achieve quantum-resistant privacy guarantees. We note, however, that the current implementation described in this work still relies on assumptions that are not quantum-secure (i.e., elliptic curve pairings). The improved design maintains the beneficial properties of the original construction (atomicity, scalability, auditability, liquidity unification, and modular regulatory compliance) while substantially reducing computational overhead and providing robust security against quantum adversaries.

Our primary contributions include:

- optimized implementation performance for anonymous inter-bank transactions that reduces computational complexity while maintaining privacy guarantees
- a transition from ECC-based to hash-based bank identification in the Enygma protocol providing quantum-resistant security, with modular components that can be upgraded as quantum-resistant technologies mature
- enhanced overall performance characteristics that better support national-scale deployment scenarios

The improved protocol achieves quantum privacy by ensuring that quantum adversaries cannot infer transaction metadata including payer identity, payee identity, or transaction amounts. The key advancement lies in replacing the ECC-based bank identification system with hash-based alternatives in the Enygma protocol, significantly improving quantum resistance. While the current implementation utilizes ZK-SNARKs for certain proof components, the modular architecture enables straightforward migration to quantum-resistant alternatives as they become available. This design philosophy balances immediate deployment feasibility with long-term security requirements, providing a practical path forward for CBDC implementations that must consider both current and future threat models.

2 System Model

2.1 Overview

Our design remains unchanged and is the same as the original work [1]. First, we assume that the central bank operates a permissioned blockchain[3]. Moreover, each commercial bank runs their own privacy ledger (i.e., private blockchain). We assume that both components run an Ethereum Virtual Machine (EVM) blockchain and that the balances of each commercial bank are private. Each bank has a post-quantum key-pair, which is used to obtain shared secrets with all the other banks, which are then used to derive symmetric keys to encrypt

[3] Our design can run on any EVM blockchain.

transaction information. The core component is each financial institution run a tailored, single-node blockchain interconnected through a central regulatory authority's blockchain (commit chain) that acts as a protocol defining standard messaging service.

2.2 System Architecture

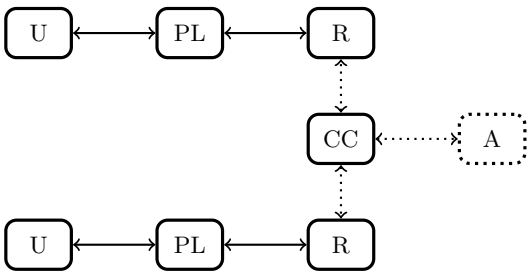

Fig. 1. System Architecture. Users bank with privacy ledgers controlled by the corresponding financial institution. Each privacy ledger uses a relayer to communicate with the commit chain. The commit chain is a layer-1 blockchain that runs smart contracts and also acts a bulletin board. Optionally, we assume the existence of an auditor to monitor the transactions that take place.

In figure 1, we describe the basic architecture of Rayls. A commit Chain (CC) as the central component of the architecture, is a semi-public chain that is being administered by the central bank and can be validated by the public. Each Privacy Ledger (PL) represents an independent financial institution with its own private internal state.

Users (U) of financial institutions are directly connected to the respective Privacy Ledger of corresponding financial institution and have a wallet address on their corresponding privacy ledger.

Each privacy ledger has a wallet address and a balance per currency on the commit chain, representing its total internal balance. Privacy ledgers are connected to the Commit Chain (CC) through Relayers (R), which provide interoperability capabilities between PL and CC.

The Admin role (Central Bank) has special privileges. For instance, a central bank issues a private token on the commit chain and distributes the balance to each PL accordingly. It is through these balances that financial institutions transact between them and the users.

The commit chain guarantees integrity and regulations between privacy ledgers. This prevents double spending and minting more tokens out of thin air through universally verifiable zero-knowledge proofs. Therefore, the central authority can guarantee authenticity of the transactions.

2.3 Assumptions

We assume that the commit chain is a blockchain with a byzantine fault tolerant (BFT) consensus. This commit chain runs a smart contract for the private transactions while also acting as a bulletin board for parties to publish encrypted information alongside the private transactions. This smart contract enforces a set of predefined rules embedded in the code. For example, a deterministic nullifier is checked when a transaction is submitted and a transaction is discarded if an entity attempts to submit two transactions for a specific time slot. We also assume that each privacy ledger has a quantum-secure key pair to be used for (post-quantum) key agreements in order to establish symmetric keys with each other. We assume the existence of a cryptographically secure hash function to ensure that nullifiers can be securely generated without the risk of collisions.

2.4 Adversarial Model

We assume an adversary $\mathcal{A}$ that wants to subvert the system. To successfully subvert the system, the adversary must be able to perform a double spend transaction, mint funds maliciously and successfully spend such funds, and/or break the privacy of the system. Breaking privacy implies inferring which parties are transacting with each other and/or the amounts of each transaction.

3 Our Design

In this paper, we propose two main approaches. First, the system architecture with the privacy ledgers. We call this Rayls. Second, we propose a new protocol for private transactions inspired in zkLedger [2]. We call this Enygma.

3.1 Formal Specification of User Algorithms

In this section we formally specify the user algorithms in the Enygma protocol. Our description largely follows the terminology in [3, Section 5], except that we add two additional algorithms: SetUp generates the shared secrets between honest parties via an Authenticated Key Exchange (AKE) protocol, and Verify allows parties to check if they have received funds (and if so, how much).

- $\mathsf{SetUp}(1^\lambda)$:
  - Each party i runs $(\mathsf{sk}_\mathsf{i}', \mathsf{pk}_\mathsf{i}') \leftarrow \mathsf{ViewKeyGen}(1^\lambda)$. Each pair of parties i, j runs the AKE protocol Π_{AKE} on inputs $(\mathsf{sk}_\mathsf{i}', \mathsf{pk}_\mathsf{j}')$ and $(\mathsf{sk}_\mathsf{j}', \mathsf{pk}_\mathsf{i}')$, respectively, resulting in shared secrets $s_{i,j} = s_{j,i}$. Party i stores $s_{i,j}$ for all $i \neq j$. Each party then computes and stores a table of (v, vG) for reasonable values v of funds. There are three possible ways to achieve this:
    * Each party computes the table locally;
    * One party computes and publishes the table, and every other party verifies a fraction of it;

  - ∗ Outsource the computation of the table.
- Each party i also runs $(\mathsf{sk}_i^*, \mathsf{pk}_i^*)) \leftarrow \mathsf{SpendKeyGen}(1^\lambda)$, where $\mathsf{pk}_i^* = \mathcal{H}(\mathsf{sk}_i^*)$. This acts as the (quantum-secure) identifier in the system and will be used in the process of spending funds.

– $\mathsf{CreateAddress}(1^\lambda)$: Run $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$. The output is $(\mathsf{sk}, \mathsf{pk})$.

– $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet} = \{\mathsf{pk}_1, \ldots, \mathsf{pk}_n\}, b_i, v)$:
Let pk_i be the public key corresponding to sk_i; if $i, j \in [n]$ (i.e., $\mathsf{pk}_i, \mathsf{pk}_j \in \mathsf{AnonSet}$), b_i its corresponding balance and v the total amount to send to pk_j, proceed as follows:

1. Obtain the latest value of the time slot identifier. For simplicity we use block time as the time slot, or epoch, hence the identifier is the block number block.
2. For all $j \in [n] \setminus \{i\}$ (below we write it as $j \neq i$, i.e., we always assume that $j \in [n]$), calculate random factor $r_j := \mathcal{H}(s_{i,j}, \mathsf{block})$. Then we define $r_i := -\sum_{j \neq i} r_j$ and $v = \sum_{j \neq i} v_j$.
3. Generate commitments

$$C_i := (-v)G + r_i H$$
$$C_j := v_j G + r_j H \quad \text{for } j \neq i$$

4. Generate a nullifier to ensure one transaction per bank per epoch (again, we use block for simplicity): $\mathsf{nf} := \mathcal{H}(sk_i, \mathsf{block})$.
5. Generate the zero-knowledge proof π. There is a single zero-knowledge proof for the entire transaction. It proves the following:
   - I have enough funds to send in this transaction (i.e., $b_i \geq v$);
   - $(C_1, \ldots, C_n)$ are well-formed (i.e., the random factors $(r_1, \ldots, r_n)$ are well-formed AND I know the spend secret key sk_i^* for the slot being debited v amount AND v is equal to the sum of the credited amounts in the commitments in this transaction;
   - The nullifier nf is well-formed (i.e., it is the result of a ZK-friendly hash using my secret key sk_i for this round and the block number: block).
6. For each $j \neq i$, encrypt transaction information using a symmetric key k_j derived from the original shared secret: $k_j \leftarrow HKDF(s_{i,j}, \mathsf{block})$. Concretely, the random factors r_j, the transferred amounts v_j, the receiving addresses per bank, and respective quantities to receive, generating ciphertexts ctxt_j.
7. Output $\mathsf{tx} := (C_1, \ldots, C_n, \mathsf{nf}, \pi)$ and (ctxt_j) (for $j \neq i$).

– $\mathsf{Verify}(\mathsf{sk}_j, \mathsf{tx})$:

1. Obtain the latest block number block and filter the events in the block.
2. Parse ctxt_j from one of the events and decrypt it using k_j, (for $j \neq i$).

3. Obtain and store r_j and v_j for the current block. Update current values locally, obtain b_j, the current balance after processing this transfer.

– $\mathsf{ReadBalance}(\mathsf{sk}_j)$:
1. Compute corresponding public key pk_j.
2. Download from smart contract $C := \mathsf{acc}[\mathsf{pk}_j]$.
3. For each (block, r_j) record stored in Verify, subtract $r_j H$ from C. Let B be the final result.
4. Using the aforementioned table, find b such that $B = bG$. Output b.

3.2 Formal Specification of the Smart Contract

The smart contract can process two types of transactions: fund and transfer.

– $\mathsf{Fund}(\mathsf{pk}_i, b)$:
If $\mathsf{acc}[\mathsf{pk}_i]$ is undefined then set $\mathsf{acc}[\mathsf{pk}_i] := b$, otherwise update $\mathsf{acc}[\mathsf{pk}_i] := \mathsf{acc}[\mathsf{pk}_i] + b$.

– $\mathsf{Transfer}(\mathsf{tx})$ where $\mathsf{tx} = (C_1, \ldots, C_n, \mathsf{nf}, \pi)$ appended by ctxt_j (for $j \neq i$):
1. Check if the commitments add up to zero: $C_1 + \cdots + C_n = 0$.
2. Check if $\mathsf{block}' \leq \mathsf{block} \leq$ *block.number* , where block' is the last registered block and *block.number* is the most recent block number queried by the contract at the time of execution.
3. Verify the zero-knowledge proof π and check if proof is up to date. Concretely, if current balance of the sender in the proof corresponds to the last finalised value of $\mathsf{acc}[\mathsf{pk}_i]$ in the contract, correspondent to block'.
4. Check if nullifier nf has been used for the current block.
5. Add the commitments $(C_1, \ldots, C_n)$ to the pending pool.
6. Tally the commitments: for $\ell = 1, \ldots, n$, update $\mathsf{acc}[\mathsf{pk}_\ell] := \mathsf{acc}[\mathsf{pk}_\ell] + C_\ell$. This step happens upon a trigger[4].

4 Security of the System

A proof of security is provided in appendix A. We note that our security notions are very closely related to Zether [3]. Concretely, our main goals are achieving privacy and overdraft safety.

5 Auditing

We describe the two types of audit that exist in our solution.

[4] This trigger could be an external service (oracle) notifying the contract that the time slot has been updated, or simply the arrival of a transaction with a higher epoch value than the previously registered one.

Universal Audit. Due to the universal verifiability of the zero-knowledge proofs, anyone can check that the proofs posted in each block are correct. Additionally, an entity that does not wish to verify the ZK proofs of Enygma, can add all the commitments and check if the result adds up to the point in infinity, thus implying that the commitments add up to zero. This addition check is performed at a smart contract level and is transparent and verifiable for anyone observing the system.

Admin Audit. Optionally, under strict auditing requirements, privacy ledgers can share with the admin the post-quantum key-pairs that result in the shared secrets used to derive the encryption keys. This sharing of secret keys allows the admin to open every transaction that takes place in the network, similarly to a 'view' key. We highlight that this key sharing process does not allow the admin to spend funds on behalf of any of the privacy ledgers as this key is only used for encryption.

6 Implementation & Performance

We implemented the complete system using Golang (Go), Solidity, and gnark. Concretely, the Privacy Ledger is a fork of Geth [4] (Go implementation of Ethereum) with a custom change of database to use MongoDB instead. This different DB allows this component to provide higher performance to end users. The commit chain is instantiated as an Hyperledger Besu permissioned EVM blockchain. The relayer is a read/write node, also written in Go, that reads events from both the privacy ledger and the commit chain and performs actions accordingly.

We use ZK-friendly cryptographic primitives in the the Enygma smart contract. Concretely, the Baby Jubjub Elliptic Curve as well as the Poseidon [5] hash function. When registering on the commit chain, for the purpose of this implementation, nodes register a CSIDH [6] public-key and perform a post-quantum key agreement with all the other nodes in the network.

Our implementation of a ZK circuit is in gnark [7] and covers the entire ZK statement we describe in our protocol description. Concretely, that the sender knows the secret key behind a set of public keys, that the index behind that public key corresponds to a debit, and that the same amount is being credited to other parties involved in the transaction. Additionally, that the commitments of the transaction are well-formed and that the random factors are derived from the pre-established shared secrets.

Our implementation results in 80053 constraints and can run on commodity hardware (e.g., a mac mini M1 from 2020). We provide benchmarks for our implementation on different machines.

We highlight that our implementation is currently being tested by a Central Bank in a CBDC pilot and is in production with a clearing house of one of the top 10 biggest nations in the world.

Machine	Mac mini	c5.4xlarge	c5.12xlarge	c5.24xlarge	
Time (ms)	4002	3414	2260	2425	Previous Work
	408	317	216	204	This work

Table 1. Performance of ZK proof generation of Enygma I and Enygma II in different machines.

6.1 Theoretical Throughput Limits

In this section, we discuss the throughput of the Enygma protocol. We note that two types of transactions exist in our system: intra-bank and inter-bank.

Intra-bank transactions are transactions that take place inside a privacy ledger and do not rely on the Enygma protocol. Inter-bank transactions, on the other hand, are transactions involving at least two privacy ledgers and can use the Enygma protocol to achieve privacy. The throughput of both is completely different and hard to precisely calculate. Note that the throughput of intra-bank transactions is constrained by the computational resources and potential gas limits available to each bank's infrastructure (i.e., privacy ledger).

In our system, we do not have a gas limit on the commit chain. Therefore, unlike traditional blockchains, our theoretical max throughput is not capped by the gas limit present in each block.

We provide a maximum theoretical throughput for both inter-bank transaction scenarios: wholesale CBDC transactions and retail CBDC transactions.

Wholesale CBDC (wCBDC). If every participant (i.e., a bank) in the network submits an Enygma transaction, using an anonymity set of k, the theoretical maximum throughput is given by:

$$n_{\text{banks}} \times (k-1) \quad \text{per block}$$

This represents pure cross-bank transfers where each bank sends one transaction to a maximum of $(k-1)$ other banks per block, suitable for wholesale CBDC scenarios where banks transact directly with each other without including any transactions belonging to end-users. In our setting, a bank can only submit one transaction per epoch (i.e., a block) due to the use of the nullifier.

Retail CBDC (rCBDC). To realize the functionalities of a retail CBDC, banks can leverage the original Enygma protocol and append some additional (encrypted) data to Enygma transactions. This additional data will effectively contain routing information belonging to the users (or clients) that are to receive the funds. As a result, Enygma transactions can effectively leverage a double batching property: a transaction can pay multiple banks at once, and each of those payments can be divided by multiple receiving clients in each bank. For

retail CBDC scenarios where banks batch end-user transactions, the throughput expands to:

$$n_{\text{banks}} \times (k-1) \times n_{\text{addr}} \quad \text{per block}$$

where n_{banks} is the number of banks in the network and n_{addr} denotes the maximum number of receiving addresses per bank.

Gas-based Throughput. In a traditional blockchain setting, where there is a limit on the gas usage per block, the throughput is:

$$n_{\text{tx}} = \frac{\text{gas limit}}{\text{tx gas cost}} \quad \text{per block}$$

We note that, for example, in Ethereum, every block has the capacity to use 30 million gas yet has a target of 15 million gas per block. Additionally, each (Groth16) proof verification in our system is consuming approximately $383,000$ gas units. Using additional data (e.g., calldata) incurs in an additional gas cost.

6.2 Balance Rollover

A naive implementation of our design, where every new transaction is automatically tallied, is not viable. This is the case because when a set of commitments is added, the next sender must know how to open their latest hidden balance, which is the previous commitment balance plus new transaction commitment. This is not feasible in a blockchain setting due to the way transactions are processed. To circumvent this issue, we rollover (tally) at the beginning of every new block, defining the block time as an epoch. Concretely, when the first transaction of the next block is processed, all the previous commitments are tallied. As a result, we have a requirement: privacy ledgers must keep track of the transactions that take place in the network and perform the balance calculation locally before submitting a transaction for the upcoming block(s).

Furthermore, this also introduces a significant challenge: the need to process all data for the last block, query the necessary information, generate proofs, and submit them to the blockchain within a single epoch timeframe. Enygma operates optimally under the following condition:

$$\frac{\Delta_\pi}{E_t} \ll 1 \ ,$$

where Δ_π represents the proof generation plus submission time and E_t denotes the epoch time, currently equivalent to one block time. Since transactions should not remain in a pending state for extended periods, it is crucial that E_t remains relatively small, on the order of a few seconds. Therefore, for optimal performance in an environment with $E_t = 5s$, for example, we require $\Delta_\pi \ll 5s$ which

is achievable on any computer manufactured after 2020. However, it is important to note that the load conditions of the relayer also affect Δ_π, and should be handled properly.

Failure to meet this condition results in the relayer frequently submitting proofs that are technically valid but based on outdated balance information. The smart contract rejects these submissions, forcing the relayer to retry transactions that would succeed if submitted with current data.

6.3 Optimizations

We now describe possible optimizations to the system.

k-anonymity. To have for a greater level of flexibility in our system, we can enforce a constant-sized anonymity ($k = 6$). This separation allows for the addition and removal of banks without requiring the deployment of a custom new ZK verifier for every update. This approach, however, impacts the anonymity of each transaction. We have implemented this feature.

Separating Enygma transactions into two ZK proofs. It is possible to tweak the protocol to use two zero-knowledge proofs instead of one. First, a proof that can contain all of the offline components: the set of commitments that comprise a transaction. Proving the correctness of these elements is the most expensive part of the computation and can be done in advance, as long as the transfer amount is defined ahead of time. The online component, is then simply proving (in zero-knowledge) the ability to open their latest on-chain balance and that they have enough funds for the transfer. This approach results in an offline component of approximately 66% of the total prover time and a real-time component of 33%. We highlight, however, that this results in an increase of transaction size (two ZK proofs) and transaction cost (verification of two ZK proofs).

7 Extensions

We now describe possible extensions to our protocol.

7.1 E-Cash

Our system can be extended to support an even more private retail CBDC, in this case running inside each privacy ledger. The privacy for each user in the normal case is simply the fact that the operator of the financial institution is expected to keep that data secure and private to external parties. We note, however, that the segregation of privacy ledgers results in a setting where users are directly connected to a financial institution, in this case the commercial bank. This architecture resembles the original e-cash [8] work where users submit a

blinded coin and the bank returns a (blindly) signed coin that can be spent privately by the user. As a result, we can use the fact that the privacy ledger effectively produces a chain of blocks to use a trust-minimized e-cash [9] approach, thus allowing users to have cash-like privacy in the transactions that take place.

8 Previous Work

Two projects are most comparable against our work: Zether [3] and zkLedger [2].

Zether uses Σ-Bullets, a combination of Σ-protocols and Bulletproofs over the ElGamal encryption scheme ('in the exponent') to ensure private bi-party transactions where the amount being transferred is private, but the entities involved in the transaction are public. The authors also introduce anonymous Zether, to ensure a bigger anonymity set. The proof size is logarithmic w.r.t the size of the set.

zkLedger uses Pedersen commitments for the transactions along with a 'Sigma' style proof construction. This approach results in linear performance for proof generation, verification time, and proof size. As a result, this approach renders zkLedger impractical for a large number of banks, especially when implementing it to run on a decentralized blockchain.

We use zkLedger as a starting point for our design due to the quantum-secure hiding nature of the Pedersen commitments. Concretely, we explore a setting where we group users into different financial institution buckets and provide a different high-performant ledger for each bucket. We then design a private transaction mechanism based on zkLedger to allow banks to privately transact with each other. This allows for higher-performance as each financial institution can provide fast transactions to their users and banks can transact among each other efficiently.

Our approach follows the rationale that the number of banks is much smaller than the number of users. Therefore, a private payment solution to be used among banks can provide good performance for the system, while also segregating the payments inside the ledger of each financial institution. This is particularly suited for the financial markets due to the inherent separation of client wallets among each institution.

8.1 Differences vs other approaches

Comparison with zkLedger zkLedger uses one ZK proof per commitment in a transaction. Our approach, on the other hand, relies on a single ZK-SNARK for the entire transaction payload. Moreover, we ensure that the random factors in the Pedersen commitments are derived from the (quantum-secure) shared secret between each privacy ledger. This correctness of the random factor is part of the zero-knowledge proof and allows for an auditor to see all the transactions that take place using a view key. zkLedger focuses primarily on cross-bank settlement and resembles our model in that aspect. The original design, however, does not have any programmability or decentralization. We, however, ensure that the validation of transactions takes place on a commit chain that is visible to anyone.

Comparison with (Anonymous) Zether Zether uses discrete-log range proofs for the transactions that take place. Zether does not provide any type of privacy against a quantum computer adversary due to the use of the ElGamal encryption scheme. Zether, however, has one advantage in its ability to read the balance at any point in time just by having the secret key. In our system, users must perform a computation and know of every received transaction to obtain the corresponding random factor from the Pedersen commitment. Zether assumes a monolithic blockchain where every user has an encrypted balance on chain. We diverge away from this model and segregate financial institutions to ensure they can individually provide high performance to their users while having the ability to make efficient 'cross-chain' transfers.

Comparison with L2 Scaling Our architecture is composed of different private blockchains, the privacy ledgers, that can effectively be considered L2's without a public data availability layer. This makes our solution similar to Plasma [10]. A Plasma chain is a separate blockchain anchored to Ethereum main net but executing transactions off-chain with its own mechanism for block validation. Plasma chains use fraud proofs to arbitrate disputes.

9 Discussion

In this section, we cover multiple discussion topics relevant to our work.

9.1 Cross-bank transfers

We expose a mechanism to allow banks to transact with each other. Extending this to transactions between clients in different banks is simple. First, the users lock funds inside their privacy ledgers. Second, the privacy ledger batches these transactions from users into a single Enygma transaction. We note that each Enygma transaction can contain many transactions as a) one transaction can pay more than one bank at once; and b) each of those bank payments can contain funds from different users. This separation is specified in the attached encrypted data.

9.2 Improving the brute-force of the balance

When receiving a transfer, the recipient obtains a commitment in the form of $vG+rH$, where v is the latest balance in the system. Due to the way the random r factor is generated, the recipient is able to obtain its inverse and remove it, thus obtaining vG. This value v, however, is not easily obtainable, due to the nature of the discrete log problem. Therefore, a user must have a mechanism to obtain v from vG. The trivial solution is to perform a brute-force for all the possible balances v. This is traditionally a reasonable approach because the balance of a bank is expected to be within reasonable brute-force ranges. In our protocol description, we propose having a precomputed table to ensure a quick lookup. We discuss three different approaches below:

Precomputation of hidden balances. A system entity can precompute a table with all the reasonable values vG for the values v that are in the expected balance range. This list can then be shared with the network and each individual party can perform a random partial check. Ideally, N parties performing a random partial check would catch any malicious values in such a list. This would ensure that the linear balance computation is only performed once. We note, however, that it's possible to improve on this linear complexity by using the baby-step giant-step approach from [11].

Ongoing table computation. Recipients can compute and store the balance table progressively. This approach results in a variable amount of work per transaction. However, it is possible to, over time, compute the balance table, perform the lookup locally first upon receiving a transaction. If the value is not present, then the recipient can start brute-forcing from the highest value in the database.

Encrypted data. In our setting, since all parties share a secret that they can use to encrypt, the sender can optionally include an encrypted payload containing additional transaction data. The recipient can decrypt the ciphertext and obtain the transfer amount.

9.3 Quantum-Security

The entire system is at least quantum private in an initial phase due to the use of a quantum-secure key agreement, cryptographically-secure hashing, and zero-knowledge proofs. Therefore, it is possible that the system can rely initially on ZK-SNARKs[12][13][14] and still preserve privacy against a quantum-adversary and, when in danger of a quantum computer, switch to post-quantum ZK proofs, such as ZK-STARKs [15] or lattice-based [16], thus ensuring end-to-end quantum-security in the system. Presently, the entire implementation is quantum-secure except for the ZK component, which uses the Groth16 proving scheme. Updating this ZK component is currently under development.

The choice of quantum-secure cryptography algorithms for the key exchange is challenging as seen by the recent devastating attacks against isogeny-based cryptograghy [17] and the recent scare against lattice-based cryptography [18]. We remark, however, that failed attempts to break the standardized lattice-based primitives should give more confidence to the community in such primitives. We, therefore, recommend implementing this system using the Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM [19]).

9.4 Trusted Setup

While our scheme is agnostic to the underlying used zero-knowledge scheme, we presently favor the use of Groth16 [12]. We highlight this specific construction due to its lightweight verifier requirements. A limitation of this scheme, however, is the requirement of a trusted setup. While frequently cited as a fundamental

problem, we emphasize that this setup can be achieved in a multi-party fashion [20], which can significantly minimize trust assumptions. This multi-party approach is particularly suited here as the system involves multiple regulated and reputable parties that are mutually suspicious. As a result, the requirement of having at least one honest party in the setup ceremony is with extremely high probability achieved.

We note that this 'problem' may disappear as the system transitions into a post-quantum ZK scheme.

10 Conclusion

We proposed a novel central bank digital currency design. We use privacy ledgers, which are a single-node private blockchain, as a segregated and contained silo for each financial institution to run their internal day-to-day operations with high-performance. This resembles the L2 scaling ideas present in Ethereum. We ensure interoperability and connectivity with other financial institutions by introducing the use of a central 'commit chain', resulting in a hub and spoke model. All of the data that flows through this commit chain is either encrypted (i.e., symmetric encryption) using keys derived from a post-quantum key agreement, cryptographically-secure hashes, and in the form of a zero-knowledge proof. This results in extremely strong security and privacy properties. All of our system is EVM-compatible and, even though we have a custom deployment of our system, is easily extendable to existing blockchains (e.g., Ethereum, Polygon, Avalanche). As a result, our proposed Enygma protocol allows for a private, scalable, quantum-ready currency.

Instead of having a global ledger for all users, we separate users into different privacy ledgers. For example, we can group hundreds of millions of users into approximately 400 privacy ledgers[5]. Our Enygma protocol then runs between the privacy ledgers as opposed to between all the users in the system resulting in a linear complexity in number of financial institutions, which is orders of magnitude smaller than the number of actual users in the system.

With this paper, we present our ideas for improving Rayls and an initial analysis of it, with the hope that our work will stimulate additional fruitful discussion, analysis, and refinements. We are actively testing our design with financial institutions and expect to improve on this design and cover potential gaps in our system.

[5] The number 400 is not arbitrary and reflects a real-world example

References

1. M. Yaksetig and J. Xu, "Rayls: A novel design for cbdcs," The 6th Workshop on Coordination of Decentralized Finance (CoDecFin) 2025, 2025.
2. N. Narula, W. Vasquez, and M. Virza, "zkledger: privacy-preserving auditing for distributed ledgers," in *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'18. USA: USENIX Association, 2018, p. 65–80.
3. B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, "Zether: Towards privacy in a smart contract world," in *Financial Cryptography and Data Security: 24th International Conference, FC 2020 , Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 423–443.
4. Ethereum, "go-ethereum," 2017. [Online]. Available: https://github.com/ethereum/go-ethereum
5. L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," Cryptology ePrint Archive, Paper 2019/458, 2019. [Online]. Available: https://eprint.iacr.org/2019/458
6. W. Castryck, T. Lange, C. Martindale, L. Panny, and J. Renes, "Csidh: An efficient post-quantum commutative group action," Cryptology ePrint Archive, Paper 2018/383, 2018. [Online]. Available: https://eprint.iacr.org/2018/383
7. Consensys, "Gnark," 2021. [Online]. Available: https://docs.gnark.consensys.io/
8. D. Chaum, A. Fiat, and M. Naor, "Untraceable electronic cash," in *Advances in Cryptology — CRYPTO' 88*, S. Goldwasser, Ed. New York, NY: Springer New York, 1990, pp. 319–327.
9. M. Yaksetig, "A trust-minimized e-cash for cryptocurrencies," Cryptology ePrint Archive, Paper 2024/444, 2024. [Online]. Available: https://eprint.iacr.org/2024/444
10. J. Poon and V. Buterin, "Plasma: Scalable autonomous smart contracts," 2017. [Online]. Available: https://plasma.io/plasma.pdf
11. D. Shanks, "Class number, a theory of factorization and genera," Proceedings of Symposium of Pure Mathematics, Vol. 20, 1971.
12. J. Groth, "On the size of pairing-based non-interactive arguments," Cryptology ePrint Archive, Paper 2016/260, 2016. [Online]. Available: https://eprint.iacr.org/2016/260
13. S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam, "Ligero: Lightweight sublinear arguments without a trusted setup," Cryptology ePrint Archive, Paper 2022/1608, 2022. [Online]. Available: https://eprint.iacr.org/2022/1608
14. A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953
15. E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," Cryptology ePrint Archive, Paper 2018/046, 2018. [Online]. Available: https://eprint.iacr.org/2018/046
16. V. Lyubashevsky, N. K. Nguyen, and M. Plancon, "Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general," Cryptology ePrint Archive, Paper 2022/284, 2022. [Online]. Available: https://eprint.iacr.org/2022/284

17. W. Castryck and T. Decru, "An efficient key recovery attack on sidh," Cryptology ePrint Archive, Paper 2022/975, 2022. [Online]. Available: https://eprint.iacr.org/2022/975
18. Y. Chen, "Quantum algorithms for lattice problems," Cryptology ePrint Archive, Paper 2024/555, 2024. [Online]. Available: https://eprint.iacr.org/2024/555
19. NIST, "Module-lattice-based key-encapsulation mechanism standard," 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf
20. V. Nikolaenko, S. Ragsdale, J. Bonneau, and D. Boneh, "Powers-of-tau to the people: Decentralizing setup ceremonies," Cryptology ePrint Archive, Paper 2022/1592, 2022.

A Security Analysis

In this section we present the formal security definitions and proofs for the Enygma protocol. Our definitions largely follow [3].

A.1 Security Definitions

We assume an adversary $\mathcal{A}$ that has full visibility over all the transactions that happen in the network and runs a full node of the commit chain. To this end, we first define a basic version of the security game, which is played between a challenger, a smart contract, and the adversary $\mathcal{A}$. At the beginning of the game, the challenger runs $\mathsf{SetUp}(1^\lambda)$, generating the key pairs, the shared secrets, and the discrete logarithm table for each party. After that, $\mathcal{A}$ can interact with the challenger and the smart contract in the following manner:

1. $\mathcal{A}$ can instruct the challenger to run any user algorithm in Section 3.1 and see the output (and the resulting transaction, if any, is sent to the smart contract), with the following exceptions:
   - $\mathcal{A}$ cannot instruct the challenger to run SetUp or Verify. (Note that the Verify algorithm requires knowledge of an honest party's secret key.)
   - If the algorithm is $\mathsf{CreateAddress}$, $\mathcal{A}$ only sees the public key pk in the output, and not the secret key sk.
   - If the algorithm is $\mathsf{CreateTransferTx}$, $\mathcal{A}$ specifies the sender's public key pk_i instead of its secret key sk_i.
2. $\mathcal{A}$ can directly send any transaction to the smart contract.
3. $\mathcal{A}$ can instruct the smart contract to process any number of pending transactions and update its state accordingly.

We now define and prove two security properties of the Enygma protocol, namely overdraft-safety and privacy.

Privacy The privacy game is the same as the basic security game, except for the following: at some point, the adversary $\mathcal{A}$ sends two consistent instructions labeled 0 and 1 to the challenger; the challenger samples a bit $b \leftarrow \{0,1\}$ and executes the b-th instruction. At the end of the game, $\mathcal{A}$ outputs a bit b' as a guess for b. $\mathcal{A}$ succeeds if $b' = b$.

Two instructions are *consistent* if they refer to the same user algorithm, and

- If it is $\mathsf{CreateTransferTx}$, then both instructions must have the same $\mathsf{AnonSet}$ field; furthermore, if either of the receivers is corrupt, then both instructions must have the same receiver and the same amount.
- If it is $\mathsf{ReadBalance}$, then both instructions must return the same value.

We say a payment mechanism is *private* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds in the privacy game is upper-bounded by $1/2 + \mathsf{negl}(\lambda)$.

Overdraft-safety. The overdraft-safety game is exactly identical to the basic security game above, and the adversary $\mathcal{A}$ succeeds if at any point a $\mathsf{ReadBalance}$ instruction results in a negative value. We say a payment mechanism is *safe against overdrafts* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds in the overdraft-safety game is at upper-bounded by $\mathsf{negl}(\lambda)$.

A.2 Security Proof

Privacy Recall that in the privacy game, the adversary $\mathcal{A}$ must issue two consistent instructions at some point, and guess which one was executed. It is straightforward that two consistent instruction generate identical views unless they are $\mathsf{CreateTransferTx}$; below we consider $\mathcal{A}$ sending two consistent $\mathsf{CreateTransferTx}$ instructions.

Let Query be the event that $\mathcal{A}$ queries $\mathcal{H}(s_{i,k}, \mathsf{block})$ for any i, k and block is the block number when $\mathcal{A}$ sends the two consistent instructions. By the security of the AKE protocol, $\Pr[\mathsf{Query}]$ is negligible. On the other hand, if Query does not occur, then $r_k = \mathcal{H}(s_{i,k}, \mathsf{block})$ is a uniformly random integer in $\mathbb{Z}_p$. with overwhelming probability $r_k = \mathcal{H}(s_{i,k}, \mathsf{block})$ is a uniformly random integer in $\mathbb{Z}_p$; that is, C_i and C_j are Pedersen commitments to $-v$ and v respectively (using independent randomnesses r_i and r_j). By the (perfectly) hiding property of Pedersen commitment, v is independent of C_i and C_j. Since v is not used anywhere other than C_i and C_j in $\mathsf{CreateTransferTx}$, we know that v is independent of $\mathcal{A}$'s view.

Furthermore, for any v, $C_1, \ldots, C_n$ are independent uniformly random group elements, so they leak no information about i (the sender) or j (the receiver). For the remaining parts of $\mathcal{A}$'s view, nf and the r' values leak no information about i or j because $s_{i,k}$ is pseudoranrom in $\mathcal{A}$'s view, and π leaks no information about i or j due to the zero-knowledge property of the ZK scheme.

Overdraft-safety. For any honest party i, let b_i be its balance, i.e., $b_i = \mathsf{ReadBalance}(\mathsf{sk}_i)$. We want to show that $b_i < 0$ with negligible probability at any time.

We first define an ideal protocol as follows: each b_i is initialized to 0, and

- For a $\mathsf{Fund}(\mathsf{pk}_i, v)$ algorithm, set $b_i := b_i + v$;
- For a $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet}, b_i, v)$ algorithm, set $b_i := b_i - v$ and $b_j := b_j + v$.

Furthermore, only honest $\mathsf{CreateTransferTx}$ algorithms are processed, i.e., all transactions tx^* from the adversary $\mathcal{A}$ are ignored.

Obviously, the probability that $b_i < 0$ at some time is negligible in the ideal protocol, due to the soundness of the zero-knowledge proof (recall that in every transaction, the proof includes the statement $b_i \geq v$). We next show that for any i and at any time, the probability that b_i in the ideal protocol does not match b_i in Enygma is negligible. These two values differ only if $\mathcal{A}$ affects a transaction in one of the following ways:

- *Malicious spending*: $\mathcal{A}$ sends to the smart contract a transaction tx^* where the tuple of commitments $(C_1^*, \ldots, C_n^*)$ is different from all honest transactions, such that the zero-knowledge proof π^* verifies.
- *Double spending*: $\mathcal{A}$ passes to the smart contract an existing honest transaction tx for algorithm $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet}, b_i, v)$, such that party i's new balance is not $b_i - v$ or party j's new balance is not $b_j + v$ (e.g., $\mathcal{A}$ makes party i send $2v$ amount of funds to party k).

We now show that $\mathcal{A}$ has negligible probability of successfully performing either of these two attacks. For malicious spending, let $\mathcal{A}$'s transaction tx^* be from party i. The zero-knowledge proof π^* verifies only if $(C_1^*, \ldots, C_n^*)$ is well-formed; in particular, this means that the $r_1, \ldots, r_n$ factors within $C_1^*, \ldots, C_n^*$ are all well-formed, and $C_i^* = (-v)G + r_i H$ (because π^* contains a proof of the statement that its sender known sk_i). The only remaining case where $(C_1^*, \ldots, C_n^*)$ is well-formed is that $\mathcal{A}$ sees some honest transaction $(C_1, \ldots, C_n)$ from the same epoch, and replaces $(C_j, C_k) = (vG + r_j H, r_k H)$ with $(r_j H, vG + r_k H)$ (i.e., changes the intended receiver from j to k).[6] However, this attack is infeasible as v is hidden from $\mathcal{A}$ (due to privacy of the protocol).

For double spending, by the definition of Verify, $(C_1, \ldots, C_n)$ implies that party i sends v amount of funds to party j; therefore, the only possible attack without changing $(C_1, \ldots, C_n)$ is that $\mathcal{A}$ copies $(C_1, \ldots, C_n)$ from some honest transaction and sends it more than once. Since we assume that an honest user can only run $\mathsf{CreateTransferTx}$ once per epoch, the block number will change during these two transactions; say they are block and block^*, where block is the block number of the honest transaction (so $\mathcal{A}$ sees $\mathsf{nl} = \mathcal{H}(r_i, \mathsf{block})$). As argued in privacy r_j is uniformly random in $\mathcal{A}$'s view with overwhelming probability, so the probability that $\mathcal{A}$ queries $\mathcal{H}(r_i, \mathsf{block}^*)$ is negligible. But if $\mathcal{A}$ does not make this query, then the correct nullifier $\mathcal{H}(r_i, \mathsf{block}^*)$ is uniformly random in $\mathcal{A}$'s view, so by the soundness of the zero-knowledge proof, the probability that $\mathcal{A}$'s nullifier is well-formed is negligible.

[6] Note that if the commitments are well-formed, all but two of them must be in the form of $r_k H$. In particular, if $\mathcal{A}$ replaces $(C_j, C_k) = (vG + r_j H, r_k H)$ with $((v-1)G + r_j H, G + r_k H)$, then three of the resulting commitments, C_i^*, C_j^*, C_k^*, will not be in that form, so the commitments will not be well-formed.